TechDella

PDF-07 · TECHDELLA STUDIOS · GUIDE

Scope the MVP you can actually *ship.*

Eight steps from idea to a brief a build team can estimate: the problem, the riskiest assumption, one core journey, a cut-down feature list, the build approach and how you will know it worked.

FOR	TIME	OUTPUT
Founders and product leads before a first build	Half a day, ideally with a co-founder or advisor	A one-page MVP brief you can send to any build team

START HERE

What an MVP is for

A minimum viable product is the smallest thing you can put in front of real users to learn whether your riskiest assumption is true. It is a learning tool first and a product second.

Most MVPs run late for the same reason: the scope grows to cover every user, every edge case and every feature a competitor has. This guide works the other way. You start from the one thing you most need to learn, and keep only what it takes to learn it.

Signs your scope is too big

- ☐ It serves more than one type of user equally (for example buyers *and* sellers *and* admins, all fully featured).
- ☐ The feature list has more than about ten "must have" items.
- ☐ You cannot say in one sentence what you will learn from launch.
- ☐ It includes features "for later" that are being built now "because it's easier".
- ☐ Nobody outside the team has used a prototype or a manual version yet.

If you ticked two or more, the steps that follow will save you time and money.

The eight steps

#	Step	Output
1	**Problem and user**	One problem statement, one primary user
2	**Riskiest assumptions**	The assumption the MVP must test
3	**Core journey**	One end-to-end flow, step by step
4	**Feature cut**	A MoSCoW list with a short "Must" column
5	**The things people forget**	Accounts, payments, data, compliance, analytics
6	**How to build it**	No-code, low-code or custom, with reasons
7	**Budget and timeline drivers**	What will make it cost more or take longer
8	**Success metrics and launch**	What "worked" means, and who sees it first

STEP 1

Problem and user

Name one primary user and one problem. Everyone else is secondary for now.

Primary user
Role, context, and what they use today instead of your product

The problem, in their words
Quote a real conversation if you have one

How often it happens and what it costs them
Time, money, risk, frustration

Current workaround
Spreadsheets, an agency, a competitor, doing nothing

Problem statement

[Primary user] needs a way to **[job to be done]** because **[problem and its cost]**. Today they **[workaround]**, which **[why that falls short]**.

Evidence so far

Evidence	How many	What you learned
Problem interviews held		
People on a waitlist		
Pre-orders, letters of intent or pilots		
Manual / concierge version run		

STEP 2

Riskiest assumptions

List what must be true for the business to work. Rate each on how uncertain it is and how bad it would be if it were wrong. The MVP tests the top one.

#	Assumption ("We believe that …")	Type	Uncertainty (1–5)	Impact if wrong (1–5)	Score
1					
2					
3					
4					
5					
6					

Types: **Desirability** (do they want it?), **Viability** (will they pay enough?), **Feasibility** (can it be built and run?), **Usability** (can they use it without help?). Score = uncertainty × impact.

Can you test it without building?

Before committing to a build, check whether a cheaper test answers the same question.

Test	Answers
Landing page with a sign-up or pre-order	Do enough people want this to act?
Clickable prototype (Figma)	Do users understand the flow? Where do they get stuck?
Concierge / manual service	Does the outcome solve the problem, before any automation?
"Wizard of Oz" (people behind a simple front end)	Will users rely on it in their real workflow?
Technical spike	Is the hard part (an integration, an AI feature, a data source) possible at all?

The assumption this MVP will test

We will know it is true when …

A number, a behavior, a timeframe

STEP 3

One core journey

Write the single path a primary user takes from first arriving to getting the value. Every screen in the MVP should sit on this path or directly support it.

#	User does	Product does	Screen / touchpoint
1			
2			
3			
4			
5			
6			
7			
8			

EXAMPLE (ILLUSTRATIVE)

A booking tool for independent physiotherapists: (1) patient opens the clinic's link, (2) picks a slot, (3) enters details and pays a deposit, (4) gets a confirmation, (5) clinic sees the booking on its calendar, (6) both get a reminder. Everything else (reporting, multi-clinic support, packages) waits.

Moments that must feel right

Mark the one or two steps where users decide whether the product is worth it. Spend design time there first.

STEP 4

Cut the feature list

List every feature anyone has suggested, then sort with MoSCoW. A feature is a "Must" only if the core journey breaks without it, or if the law requires it.

Must have Journey breaks without it	**Should have** Painful to skip, but there is a workaround	**Could have** Nice; add if time allows	**Won't have (this time)** Written down so it stops coming up

Questions that cut scope

- ☐ Could an admin do this by hand for the first 50 users?
- ☐ Could an existing tool (Stripe Checkout, a scheduling tool, a form builder, email) do it instead of custom code?
- ☐ Does it serve the primary user, or someone secondary?
- ☐ Would the learning goal from step 2 change if we left it out?
- ☐ Is it needed on day one, or once there are paying customers?

STEP 5

The things people forget

These rarely appear in the first feature list, and they often decide the timeline. Tick what applies and note the decision.

Area	Needed?	Decision / notes
Accounts and sign-in Email + password, magic link, Google / Microsoft sign-in, roles	☐	
Payments One-off, subscription, invoices, refunds, taxes. Tip: a hosted checkout keeps card details off your servers and shrinks your PCI DSS scope.	☐	
Admin tools How your team views users, fixes data, issues refunds	☐	
Notifications Email, SMS, push, in-app	☐	
Integrations Calendars, CRMs, accounting, third-party APIs	☐	
AI features Which model provider, cost per use, fallback when it fails, human review	☐	
Data and privacy What personal data you store, where, for how long; privacy policy; data-processing agreements	☐	
Regulated areas Health, finance, children's data, payments: get advice early	☐	
Accessibility Keyboard use, contrast, screen readers (WCAG 2.2 AA is a common target)	☐	
Analytics The events that prove or disprove your assumption	☐	
Platforms Web, iOS, Android; app-store review time	☐	

STEP 6

How to build it

Pick the approach that tests your assumption fastest without blocking what comes next. There is no single right answer; the trade-offs are below.

Approach	Good when	Watch out for
No-code Website builders, form and automation tools, no-code app builders	You are testing demand or a workflow; logic is simple; speed matters most.	Platform limits, per-user pricing as you grow, hard migrations later, data residency.
Low-code Backend-as-a-service plus a custom front end	You need a custom experience but standard back-end pieces (auth, database, storage).	Vendor lock-in for data and auth; pricing at scale.
Custom code	The product's value is in the technology itself, you need deep integrations, or you are in a regulated space.	Higher upfront cost and time; you need a team to maintain it.
Hybrid	Custom where it matters (the core journey), off-the-shelf for the rest (payments, email, analytics, admin).	Too many tools stitched together without an owner.

Decide

- ☐ What must the MVP prove, and can a no-code version prove it?
- ☐ Who owns the code, designs and accounts at the end? (Put it in the contract.)
- ☐ What happens if the MVP works: can it carry the first paying customers, or is a rebuild planned?
- ☐ Who maintains it after launch, and how are bugs handled?

Our approach and why

STEP 7

Budget and timeline drivers

Builds cost more and take longer for predictable reasons. Know which apply before you ask for an estimate, so quotes are comparable.

Driver	Effect	Applies?
Number of user types with their own screens	Each one adds flows, permissions and testing.	☐
Native mobile apps (vs. web)	Separate builds or a cross-platform framework, plus app-store review.	☐
Third-party integrations	Each API has its own quirks, limits and approval process.	☐
Real-time features	Chat, live updates and collaboration need more infrastructure.	☐
AI features	Prompt design, evaluation, cost control and fallbacks take time to get right.	☐
Regulated data	Security reviews, hosting choices and legal sign-off.	☐
Design maturity	Starting from a brand and design system is faster than designing from scratch.	☐
Decision speed	Slow feedback from the founder side is the most common cause of delay.	☐

GET A BALLPARK

Our MVP calculator gives a rough estimate from your answers in a few minutes: **studios.techdella.com/mvp-calculator/**. Treat it as a starting point for a conversation, not a quote.

Budget range you can commit

Date that matters, and why (a pitch, a pilot, a season)

STEP 8

Success metrics and launch

Decide before launch what result means "continue", "change direction" or "stop". Otherwise any result looks like success.

Metric	Why it matters	Target	By when
Activation Share of sign-ups who complete the core journey once			
Retention Share who come back and do it again			
Willingness to pay Conversions to paid, pre-orders, pilots			
Qualitative Interviews after first use			

Launch plan

- [ ] The first 10 to 50 users are named people you can talk to, not anonymous traffic.
- [ ] Analytics events for the core journey are firing and tested.
- [ ] A feedback channel is visible in the product (an email, a chat, a short form).
- [ ] A weekly review is booked for the first eight weeks after launch.
- [ ] The "Won't have" list from step 4 is ready for the next round of decisions.

SEND IT

Your one-page MVP brief

Copy your answers here. This is what a build team needs to give you a useful estimate.

Product in one sentence	
Primary user and problem (step 1)	
Assumption to test (step 2)	
Core journey (step 3)	
Must-have features (step 4)	
Constraints: accounts, payments, data, compliance (step 5)	
Build approach (step 6)	
Budget range and key date (step 7)	
Success metrics (step 8)	
Existing assets Brand, designs, code, research	

Ready to build *yours?*

TechDella Studios designs and builds MVPs, websites and AI automation. Send us your brief and we will reply with questions, a suggested scope and next steps.

Start a project brief

studios.techdella.com/start/ · studios.techdella.com/mvp-calculator/

Sources

1. W3C, "Web Content Accessibility Guidelines (WCAG) 2.2". w3.org/TR/WCAG22/
2. PCI Security Standards Council, "PCI DSS" document library. pcisecuritystandards.org/document_library/
3. Stripe, "Integration security guide" (how hosted payment pages affect PCI scope). docs.stripe.com/security/guide